AI 활용 프롬프트엔지니어링의 발전 구조

프롬프트·컨텍스트·에이전트·MCP·하네스·루프·바이브 코딩 종합 강의자료

대화 내용 전체 보존본

박중희(연세대 심리학과)

목차

문서 구성 안내

대화 원문: 사용자 요청

AI 활용 엔지니어링의 발전 구조

0. 전체 개념 지도
1. 프롬프트의 개념
2. 프롬프트 기법의 변화
3. 하네스 엔지니어링
4. 루프 엔지니어링
5. 바이브 코딩에서의 엔지니어링 기법
6. Cursor, VS Code, Antigravity 의 활용 방법
7. 바이브 코딩을 실제 엔지니어링으로 만드는 운영 규칙
8. 수업용 4 단계 실습 과정
9. 학생 AI 활용 평가 기준
10. 최종 정리

온라인 출처 목록

※본 문서는 대화에서 생성된 전체 내용과 코드·표·출처를 보존한 강의자료이다.

문서 구성 안내

본 문서는 이 대화에서 요청되고 작성된 내용을 누락 없이 보존하기 위한 DOCX 강의자료이다. 다음 내용을 포함한다.

1. 사용자의 최초 요청문
2. 자료 작성 방향에 관한 응답 메모
3. 프롬프트 엔지니어링에서 루프 엔지니어링까지의 전체 본문
4. MCP, 하네스, 에이전트, 바이브 코딩 실습 코드와 템플릿
5. Cursor, VS Code, Google Antigravity 활용 방법
6. 수업용 실습 과정과 학생 AI 활용 평가 기준
7. 본문에서 사용된 온라인 출처 목록

대화 원문: 사용자 요청

> 프롬프트 엔지니어링, 에이전트 엔지니어링, 하네스 엔지니어링, 루프 엔지니어링
> 정확하지 않지만 이렇게 프롬프트 기법과 발전해오는 다양한 AI 활용과 MCP, 에이전트 등에 대한 상세한 자료를 분석해줘
> 1. 프롬프트에 대한 개념
> 2. 프롬프트기법의 변화
> 3. 하네스 엔지니어링의 개념과 연습방법, 실습
> 4. 루프엔지니어링 개념과 기법 실습
> 5. 바이브 코딩에서의 엔지니어링 기법
> - 다양한 개발의 요청방법, 어떤 식으로 시작하여 개발을 하는가?
> - cursor, vscode, antigravity 등...

작성 방향 메모 1

먼저 공식 문서와 실제 용례로 용어의 표준성을 검증하고, 프롬프트에서 에이전트·하네스·실행 루프·바이브 코딩으로 이어지는 발전 체계와 MCP·도구별 실습을 연결하겠습니다.

작성 방향 메모 2

다만 이는 공식적인 세대 교체가 아니라 서로 다른 설계 층을 교육적으로 배열한 것입니다. 프롬프트는 지시, 에이전트는 행위자, 하네스는 통제 환경, 루프는 반복·검증, MCP 는 도구 연결 인터페이스로 구분하겠습니다.

AI 활용 엔지니어링의 발전 구조

프롬프트 → 컨텍스트 → 워크플로 → 에이전트 → 하네스 → 루프 → 바이브 코딩

아래 내용은 **2026 년 7 월 기준**으로 정리한 것이다. 먼저 용어의 정확성부터 구분할 필요가 있다.

- **프롬프트 엔지니어링**은 이미 정착된 개념이다.
- **컨텍스트 엔지니어링**은 프롬프트를 넘어 모델에 어떤 정보·기억·도구 설명을 제공할지를 설계하는 개념이다.
- **에이전트 엔지니어링**은 모델이 다음 행동과 사용할 도구를 스스로 선택하도록 설계하는 실무 영역이다.
- **하네스 엔지니어링**은 에이전트가 실제 환경에서 안전하고 지속적으로 일하도록 실행 환경, 권한, 상태, 검증, 관찰, 복구 장치를 설계하는 개념이다.
- **루프 엔지니어링**은 최근 사용이 확대되는 실무 용어로, 에이전트가 목표에 도달할 때까지 실행–관찰–평가–수정을 반복하도록 만드는 상위 운영 설계이다.

따라서 이 개념들은 앞의 것이 사라지고 뒤의 것으로 대체되는 직선적 발전이라기보다, **설계 대상의 범위가 확장되는 과정**으로 보아야 한다. 특히 MCP 는 발전 단계가 아니라, 에이전트와 외부 시스템을 연결하는 **수평적 연결 규격**이다. 프롬프트 엔지니어링은 비교적 확립된 영역이지만, 하네스와 루프 엔지니어링은 2026 년 현재 빠르게 정리되고 있는 실무 개념이라는 점도 구분해야 한다. (OpenAI, Prompt engineering)

0. 전체 개념 지도

```
사용자의 목표
    ↓
[프롬프트 엔지니어링]
이번 요청에서 무엇을 해야 하는가?
    ↓
[컨텍스트 엔지니어링]
이 일을 하기 위해 지금 무엇을 알아야 하는가?
    ↓
[워크플로 엔지니어링]
어떤 순서로 처리할 것인가?
    ↓
[에이전트 엔지니어링]
다음 행동과 사용할 도구를 누가 결정할 것인가?
    ↓
[MCP / Tool Engineering]
외부 파일, DB, 브라우저, API, GitHub 에 어떻게 접근할 것인가?
    ↓
[하네스 엔지니어링]
어떤 환경·권한·기억·검증·복구 장치 안에서 실행할 것인가?
    ↓
[루프 엔지니어링]
언제 반복하고, 언제 멈추며, 언제 사람에게 넘길 것인가?
    ↓
[바이브 코딩]
사람이 자연어로 의도를 전달하고 위 시스템을 통해 제품을 개발한다.
```

핵심 용어의 차이

용어	설계 대상	핵심 질문
Prompt	한 번의 모델 요청	무엇을 해야 하는가?
Context	모델이 현재 참조하는 정보	무엇을 알아야 하는가?
Skill	재사용 가능한 절차적 지식	어떻게 처리하는가?
Tool	실제 실행 능력	무엇을 실행할 수 있는가?
MCP	도구·자료·프롬프트 연결 규격	외부 시스템과 어떻게 연결하는가?
Workflow	미리 정해진 처리 경로	어떤 순서로 진행하는가?
Agent	경로와 도구를 선택하는 실행 주체	다음 행동을 어떻게 결정하는가?
Harness	에이전트를 감싸는 운영 환경	어떻게 안전하고 검증 가능하게 운영하는가?
Loop	반복 실행 정책	언제 재시도·수정·중단하는가?
Eval	성능 측정 체계	결과가 실제로 좋은가?

한 문장으로 정리하면 다음과 같다.

```
프롬프트는 모델에게 일을 설명한다.
컨텍스트는 모델이 일을 이해하게 한다.
도구와 MCP 는 모델이 실제 행동을 하게 한다.
에이전트는 다음 행동을 결정한다.
하네스는 그 행동을 안전하고 검증 가능하게 만든다.
루프는 목표가 충족될 때까지 행동과 검증을 반복한다.
```

1. 프롬프트의 개념

1.1 프롬프트란 무엇인가

프롬프트에는 두 가지 정의가 혼재한다.

좁은 의미

사용자가 모델에 전달하는 질문이나 지시문이다.

```
"이 문서를 요약해줘."
"React 로 로그인 화면을 만들어줘."
```

넓은 의미

모델의 응답에 영향을 주는 전체 입력 묶음이다.

```
- 시스템 지침
- 개발자 또는 프로젝트 지침
- 사용자 요청
- 제공된 문서와 코드
- 이전 대화 내용
- 검색·RAG 결과
- 도구의 이름과 설명
- 출력 형식
- 예시
- 현재 상태와 메모리
```

실무적인 프롬프트 엔지니어링은 단순히 "말을 잘 쓰는 방법"이 아니다. **목표, 입력, 제약, 절차, 출력 형식, 성공 기준을 모델이 오해하지 않도록 표현하는 인터페이스 설계**이다. OpenAI 와 Anthropic 의 현재 지침도 명확한 요구사항과 성공 기준, 테스트 사례를 먼저 정의한 후 프롬프트를 최적화하는 접근을 강조한다. (OpenAI, Prompt engineering)

1.2 좋은 프롬프트의 구성 요소

좋은 프롬프트는 일반적으로 다음의 10 개 요소를 가진다.

```
1. 역할(Role)
2. 목표(Objective)
3. 사용자 또는 대상(Audience)
4. 입력 자료(Input)
5. 맥락(Context)
6. 범위와 비범위(Scope / Non-goals)
7. 제약조건(Constraints)
8. 작업 절차(Process)
9. 출력 형식(Output contract)
10. 완료 및 검증 기준(Acceptance criteria)
```

범용 프롬프트 템플릿

```
# 역할
당신은 [전문 역할]이다.

# 목표
이번 작업의 목표는 [측정 가능한 목표]이다.
```

```
# 대상
결과물의 주된 사용자는 [대상 사용자]이다.

# 입력 자료
다음 자료만을 근거로 작업한다.
- [자료 1]
- [자료 2]
- [코드 또는 파일]

# 현재 맥락
- 현재 상태:
- 이미 결정된 사항:
- 알려진 문제:
- 선행 작업:

# 범위
이번 작업에 포함되는 것:
1.
2.
3.

# 비범위
이번 작업에서 하지 않는 것:
1.
2.

# 제약조건
- 사용 기술:
- 수정 가능한 파일:
- 수정 금지 영역:
- 보안 요구:
- 외부 라이브러리 추가 조건:
- 성능 또는 접근성 조건:

# 작업 방식
1. 먼저 현재 상태를 분석한다.
2. 불확실한 부분과 가정을 명시한다.
3. 구현 전에 계획을 제시한다.
4. 가장 작은 단위로 구현한다.
5. 테스트와 검증을 수행한다.
6. 실패하면 원인을 분류한 후 수정한다.

# 출력 형식
다음 순서로 보고한다.
1. 현재 상태 분석
2. 구현 계획
3. 변경 파일
4. 구현 결과
5. 테스트 결과
6. 남은 위험

# 완료 기준
다음 조건을 모두 만족할 때만 완료로 판단한다.
- [기능 기준]
- [테스트 기준]
- [성능 기준]
- [보안 기준]
- [문서화 기준]

# 불확실성 처리
근거가 없으면 추측하지 말고 "확인 필요"로 표시한다.
파괴적 변경이나 범위 확대가 필요하면 실행 전에 중단하고 보고한다.

# 사고 결과 보고
내부 사고과정 전체를 출력하지 말고,
검토 가능한 계획, 가정, 선택 근거, 테스트 증거만 간결하게 보고한다.
```

1.3 나쁜 프롬프트와 좋은 프롬프트의 차이

좋지 않은 요청

```
학생 관리 앱을 예쁘고 최신 기술로 만들어줘.
```

이 요청에는 다음 정보가 없다.

- 사용자가 누구인지
- 어떤 문제를 해결하는지
- 필수 기능이 무엇인지
- 어떤 기술을 사용할지
- 데이터는 어디에 저장할지
- 완료를 어떻게 판단할지
- 테스트를 어떻게 할지

개선된 요청

```
# 목표
대학원 수업에서 학생별 과제 제출, AI 활용 기록, 교수자 평가를 관리하는
로컬 웹 애플리케이션의 MVP 를 개발한다.

# 사용자
- 교수자
- 조교
- 학생

# MVP 기능
1. 학생 등록 및 조회
2. 과제 등록
3. 학생별 제출물 업로드
4. AI 활용 과정 기록
5. 루브릭 기반 평가
6. 학생별 총점 조회

# 기술
- Frontend: React + TypeScript
- Backend: FastAPI
- Database: SQLite
- 테스트: Vitest, Pytest
- 로컬 실행을 우선하며 클라우드 배포는 이번 범위에서 제외한다.

# 비범위
- 소셜 로그인
- 결제
- 모바일 네이티브 앱
- 실시간 채팅

# 작업 순서
1. 기존 저장소가 있으면 먼저 구조와 패턴을 조사한다.
2. 곧바로 코딩하지 말고 요구사항 누락과 위험을 분석한다.
3. 데이터 모델과 API 계약을 먼저 작성한다.
4. 학생 등록부터 DB, API, UI, 테스트가 연결되는 하나의 수직 기능을 구현한다.
5. 해당 기능 검증 후 다음 기능으로 진행한다.

# 완료 기준
- 학생을 등록하고 목록에서 확인할 수 있다.
- 중복 학번은 저장되지 않는다.
- 잘못된 입력에 사용자 친화적 오류가 표시된다.
- 백엔드와 프론트엔드 테스트가 통과한다.
- 설치 및 실행 방법이 README 에 기록된다.
```

두 프롬프트의 차이는 문장의 길이가 아니라 **판정 가능한 계약이 존재하는가**이다.

1.4 프롬프트 엔지니어링의 핵심은 평가이다

프롬프트가 좋은지는 읽어보고 판단하는 것이 아니라, 반복 가능한 사례로 측정해야 한다.

```
성공 기준 정의
    ↓
평가 사례 작성
    ↓
기준 프롬프트 실행
    ↓
실패 유형 분류
    ↓
프롬프트의 한 요소만 수정
    ↓
전체 사례 재실행
    ↓
회귀 여부 확인
    ↓
프롬프트 버전 기록
```

프롬프트 평가표 예시

평가 항목	설명	배점
정확성	입력 내용과 사실을 정확히 반영하는가	30
완전성	필수 요구사항을 빠뜨리지 않았는가	20
형식 준수	지정한 구조와 스키마를 따르는가	15
근거성	추측과 근거를 구분하는가	15
안전성	민감정보·파괴적 행동을 통제하는가	10
일관성	여러 번 실행해도 품질 변동이 작은가	10

---

2. 프롬프트 기법의 변화

2.1 변화의 전체 흐름

단계	중심 기법	변화된 설계 대상
1	단순 질의	문장
2	역할·제약·출력 형식	지시 구조
3	Zero-shot·Few-shot	예시와 패턴
4	문제 분해·계획	추론 절차
5	RAG·검색·메모리	컨텍스트
6	Tool calling·ReAct	행동
7	Chaining·Routing·Parallel	워크플로
8	Agent·Multi-agent	자율적 의사결정
9	MCP	외부 시스템 연결
10	Harness	실행·상태·검증·통제
11	Loop	반복·평가·중단·복구

Chain-of-Thought, ReAct, Self-Refine, Reflexion 과 같은 연구들은 각각 문제를 단계적으로 처리하고, 외부 도구를 사용하며, 결과를 비평하고 다음 시도에 반영하는 방향으로 발전해 왔다. 다만 실제 서비스에서는 모델의 내부 사고 전체를 요구하기보다, **계획·가정·근거·검증 결과를 외부화**하는 방식이 더 관리 가능하다. (Wei et al., Chain-of-Thought Prompting)

2.2 대표적인 프롬프트·워크플로 기법

① 구조화 프롬프트

제목, 구분자, XML 태그, Markdown 구조를 이용해 서로 다른 종류의 정보를 분리한다.

```
<goal>
논문의 핵심 연구모형을 분석한다.
</goal>

<source>
여기에 논문 본문
</source>

<constraints>
- 본문에 없는 내용을 추측하지 않는다.
- 직접 확인할 수 없는 것은 확인 불가로 표시한다.
</constraints>

<output>
1. 연구문제
2. 변수
3. 가설
4. 분석방법
5. 결과
```

```
6. 한계
</output>
```

② Few-shot

설명보다 완성된 예시가 더 효과적인 작업에 사용한다.

```
입력: "매우 만족한다."
출력: {"sentiment": "positive", "score": 5}

입력: "기대보다는 별로였다."
출력: {"sentiment": "negative", "score": 2}

입력: "보통이다."
출력:
```

③ 문제 분해

복잡한 작업을 작은 단계로 나눈다.

```
한 번에 전체 결과를 생성하지 말고 다음 순서로 처리하라.

1. 요구사항 추출
2. 누락된 조건 탐색
3. 용어 정의
4. 데이터 구조 설계
5. 구현 계획
6. 구현
7. 검증
```

④ 프롬프트 체이닝

각 단계의 출력이 다음 단계의 입력이 된다.

```
요구사항 분석
→ 데이터 모델
→ API 계약
→ UI 설계
→ 구현
→ 테스트
```

⑤ 라우팅

작업 종류에 따라 다른 프롬프트나 에이전트로 전달한다.

```
질문 유형 판별
├─ 논문 분석 → 연구분석 에이전트
├─ 통계 분석 → 통계 에이전트
├─ 코드 오류 → 디버깅 에이전트
└─ 일반 질문 → 일반 응답 에이전트
```

⑥ 병렬화

서로 독립적인 작업을 동시에 처리한다.

```
에이전트 A: 보안 문제 검토
에이전트 B: UI 접근성 검토
에이전트 C: 테스트 누락 검토
에이전트 D: 성능 병목 검토
                ↓
          결과 통합·우선순위화
```

⑦ 오케스트레이터-워커

상위 에이전트가 문제를 분해하고 하위 에이전트에게 할당한다.

⑧ 평가자-개선자

한 모델 또는 에이전트가 결과를 만들고, 다른 평가자가 기준에 따라 비평한 후 수정한다.

Anthropic 은 실용적인 에이전트 시스템에서 프롬프트 체이닝, 라우팅, 병렬화, 오케스트레이터-워커, 평가자-개선자 패턴을 구분하며, 가능한 경우 복잡한 에이전트보다 단순하고 예측 가능한 워크플로부터 시작할 것을 권고한다. (Anthropic, Building effective agents)

2.3 컨텍스트 엔지니어링

프롬프트 엔지니어링이 "무엇을 지시할 것인가"에 집중한다면, 컨텍스트 엔지니어링은 "모델이 판단할 때 무엇을 보게 할 것인가"를 설계한다.

```
Prompt Engineering
= 지시를 최적화한다.

Context Engineering
= 판단에 필요한 정보를 선택·배치·압축·갱신한다.
```

모델의 컨텍스트 창은 무한한 지식창고가 아니라 제한된 작업 공간이다. 모든 파일과 모든 대화를 한꺼번에 넣으면 정보량은 증가하지만, 중요한 정보의 검색 정확도와 집중력은 오히려 저하될 수 있다. 따라서 현재 목표에 필요한 최소한의 충분한 정보를 제공하고, 장기 작업에서는 요약·압축·외부 상태 파일·검색 기반 재주입을 사용해야 한다. (Anthropic, Effective context engineering for AI agents)

좋은 컨텍스트 패킷

```
# 현재 목표
사용자 등록 API 의 중복 이메일 오류를 수정한다.

# 현재 상태
- POST /users 는 구현되어 있다.
- DB 에는 email unique index 가 있다.
- 중복 이메일 요청 시 현재 500 오류가 발생한다.

# 관련 파일
- backend/routes/users.py
- backend/services/user_service.py
- backend/models/user.py
- tests/test_users.py

# 시스템 불변조건
- API 오류 형식은 {"code", "message", "details"}이다.
- 기존 성공 응답 스키마를 변경하지 않는다.

# 알려진 실패
pytest tests/test_users.py::test_duplicate_email
예상: 409
실제: 500

# 완료 조건
- 중복 이메일은 409 를 반환한다.
- 오류 형식을 준수한다.
- 기존 사용자 생성 테스트가 통과한다.
```

컨텍스트 설계의 6 가지 원칙

```
1. Select  : 필요한 정보만 선택한다.
2. Structure: 역할별로 구조화한다.
3. Prioritize: 중요한 정보를 앞과 가까운 위치에 둔다.
4. Compress : 오래된 정보는 요약한다.
5. Refresh  : 변경된 코드와 상태를 갱신한다.
6. Persist  : 중요한 결정과 진행상태는 대화 밖에 저장한다.
```

2.4 에이전트 엔지니어링

워크플로와 에이전트의 차이

```
워크플로:
사람이 처리 순서를 미리 정한다.

에이전트:
모델이 관찰 결과에 따라 다음 단계와 도구를 선택한다.
```

예를 들어 다음은 워크플로이다.

```
문서 읽기 → 요약 → 키워드 추출 → DB 저장
```

다음은 에이전트이다.

```
목표를 확인한다.
→ 필요한 파일을 탐색한다.
→ 부족하면 검색 도구를 사용한다.
→ 코드를 수정한다.
→ 테스트를 실행한다.
→ 실패 원인을 보고 다음 행동을 선택한다.
→ 완료 기준을 충족하면 종료한다.
```

에이전트의 기본 구조는 다음과 같다.

```
관찰(Observe)
   ↓
판단·계획(Decide / Plan)
   ↓
행동(Act)
   ↓
도구 결과·환경 변화(Observation)
   ↓
검증(Evaluate)
   ↓
상태 갱신(Update State)
   ↓
계속 / 종료 / 사람에게 요청
```

OpenAI Agents SDK 역시 모델 호출 후 최종 응답, 도구 호출, 핸드오프 여부를 판단하고, 도구 결과를 다시 모델에 전달하는 실행 루프를 기본 구조로 설명한다. Anthropic 은 에이전트를 모델이 도구 사용과 진행 방향을 동적으로 결정하는 시스템으로 구분한다. (OpenAI Agents SDK, Running agents)

에이전트 설계 명세

```
agent:
  name: repository_debugger

  objective:
    description: "재현 가능한 소프트웨어 오류를 최소 변경으로 수정한다."
    success_criteria:
      - "실패 테스트가 통과한다."
```

```
    - "전체 회귀 테스트가 통과한다."
    - "공개 API 계약이 변경되지 않는다."

observations:
  - repository_files
  - git_diff
  - test_output
  - compiler_output
  - browser_console
  - application_logs

actions:
  - search_code
  - read_file
  - edit_file
  - run_test
  - run_lint
  - run_build

restrictions:
  - "운영환경에 배포하지 않는다."
  - "비밀키를 읽거나 출력하지 않는다."
  - "승인 없이 의존성을 추가하지 않는다."
  - "한 반복에서 하나의 가설만 검증한다."

memory:
  state_file: "docs/progress.md"
  decisions: "docs/decisions/"
  source_of_truth: "docs/product-spec.md"

stop_conditions:
  success:
    - "모든 완료 기준 통과"
  failure:
    - "동일 실패가 2 회 반복"
    - "최대 5 회 반복"
    - "파괴적 변경 필요"
    - "요구사항이 상충함"

escalation:
  - "DB 마이그레이션 필요"
  - "보안 정책 변경 필요"
  - "기존 API 호환성 파괴 필요"
```

에이전트 엔지니어링의 핵심 항목

항목	설계 질문
Goal	무엇을 달성해야 하는가
State	현재 어디까지 진행했는가
Observation	무엇을 관찰할 수 있는가
Tool	무엇을 실행할 수 있는가
Policy	다음 행동을 어떻게 선택하는가
Memory	어떤 정보를 다음 실행까지 보존하는가
Validator	결과를 무엇으로 검증하는가
Stop	언제 멈추는가
Budget	시간·반복·비용 한도는 얼마인가
Escalation	언제 사람에게 넘기는가

2.5 MCP 의 정확한 개념

MCP 는 **Model Context Protocol** 이다. AI 애플리케이션이 외부 데이터와 실행 기능에 접근할 수 있도록 하는 개방형 연결 규격이다.

```
MCP 는 모델이 아니다.
MCP 는 에이전트가 아니다.
MCP 자체가 추론하거나 판단하지 않는다.

MCP 는 AI 애플리케이션과 외부 시스템을 연결하는 규격이다.
```

MCP 구조

```
[Host]
Cursor / VS Code / Antigravity / AI Application
        ↓
[MCP Client]
서버 연결과 메시지 교환 관리
        ↓
[MCP Server]
Tools / Resources / Prompts 제공
        ↓
GitHub / File System / Browser / DB / API / Notion
```

MCP 서버는 주로 다음을 제공한다.

구성요소	의미
Tools	모델이 호출할 수 있는 실행 기능
Resources	모델이 읽을 수 있는 데이터와 문서
Prompts	재사용 가능한 작업 템플릿

공식 MCP 문서는 MCP 를 AI 애플리케이션과 외부 시스템을 연결하는 개방형 표준으로 설명하며, 도구는 명시적 이름과 입력 스키마를 통해 모델에 노출된다. (Model Context Protocol, Introduction)

MCP 와 관련 개념의 차이

```
MCP
= 연결 규격

Agent SDK
= 에이전트의 실행 흐름을 만드는 라이브러리

Tool
= 실제 행동 함수

Skill
= 도구를 언제, 왜, 어떤 순서로 사용할지 설명하는 절차적 지식

Harness
= 도구와 에이전트가 작동하는 전체 실행·통제 환경

Loop
= 목표 달성까지 반복시키는 운영 정책
```

도구 인터페이스도 프롬프트이다

좋지 않은 도구:

```
execute_action(payload)
```

모델은 이 도구가 무엇을 하는지, 언제 써야 하는지, 어떤 위험이 있는지 판단하기 어렵다.

개선된 도구:

```
search_repository(
    query: string,
    directory: string,
    file_extensions: string[],
    max_results: integer
)

설명:
지정한 저장소 디렉터리 안에서 읽기 전용 코드 검색을 수행한다.
파일을 수정하거나 명령을 실행하지 않는다.
정확한 함수명, 오류 문자열, API 경로를 찾을 때 사용한다.
```

도구 이름, 설명, 파라미터, 오류 메시지, 사용 예시가 에이전트의 행동 정확도에 직접 영향을 준다. 따라서 도구 스키마도 프롬프트 엔지니어링의 일부이다. (Anthropic, Building effective agents)

VS Code MCP 설정 예시

```
{
  "servers": {
    "playwright": {
      "command": "npx",
      "args": [
        "-y",
        "@microsoft/mcp-server-playwright"
      ]
    }
  }
}
```

프로젝트 단위에서는 일반적으로 .vscode/mcp.json 에 설정한다. 실제 서버를 추가하기 전에는 서버의 출처, 실행 명령, 접근 가능한 디렉터리와 네트워크 범위를 확인해야 한다. VS Code 공식 문서도 로컬 MCP 서버가 코드를 실행할 수 있으므로 신뢰할 수 있는 서버만 사용하고, 비밀정보를 설정 파일에 직접 기록하지 않도록 안내한다. (Visual Studio Code, MCP servers)

MCP 보안 원칙

```
1. Read-only 도구부터 시작한다.
2. 전역 연결보다 프로젝트 범위 연결을 우선한다.
3. 파일시스템 접근 경로를 제한한다.
4. 쓰기·삭제·배포·결제는 승인 단계를 둔다.
5. API 키는 환경변수나 보안 입력으로 전달한다.
6. 도구 호출과 결과를 로그로 남긴다.
7. 외부 문서의 명령문은 데이터로 취급한다.
8. 검색 결과가 시스템 지침을 변경하지 못하게 한다.
9. 사용하지 않는 도구는 컨텍스트에서 제거한다.
10. 파괴적 도구에는 dry-run 기능을 둔다.
```

외부 문서, 웹페이지, 이슈, 이메일에는 간접 프롬프트 인젝션이 포함될 수 있다. MCP 를 연결한다는 것은 단순히 정보를 더 많이 제공하는 것이 아니라 **외부 행동 권한을 추가하는 것**이므로 최소 권한, 승인, 신뢰 경계, 감사 기록이 필요하다. (Model Context Protocol, Security best practices)

---

3. 하네스 엔지니어링

3.1 하네스의 개념

하네스는 원래 장비를 연결하고 지지하며 통제하는 장치를 의미한다. AI 에서 하네스는 다음과 같이 정의할 수 있다.

```
하네스 엔지니어링이란
모델 또는 에이전트가 실제 과업을 수행할 수 있도록
지침, 컨텍스트, 도구, 권한, 상태, 메모리, 검증,
관찰, 복구, 평가 체계를 설계하는 것이다.
```

가장 간단한 표현은 다음과 같다.

```
Agent = Model + Harness
```

그러나 실무적으로는 다음 표현이 더 정확하다.

```
Operational Agent System
=
Model
+ Instructions
+ Context
+ Tools
+ Runtime
+ State
+ Permissions
+ Feedback
+ Evaluation
+ Human Oversight
```

Anthropic 은 에이전트 하네스를 모델 입력 처리, 도구 호출 오케스트레이션, 상태 유지, 결과 반환을 담당하는 스캐폴드로 설명한다. 또한 에이전트의 성능을 평가할 때 모델만이 아니라 **모델과 하네스의 조합**을 평가해야 한다고 구분한다. (Anthropic, Demystifying evals for AI agents)

3.2 프롬프트, 에이전트, 하네스의 차이

```
프롬프트:
"무엇을 하라"라고 설명한다.

에이전트:
"다음에 무엇을 할지" 결정한다.

하네스:
에이전트가 무엇을 보고,
무엇을 실행할 수 있으며,
어떻게 검증하고,
실패했을 때 어떻게 복구할지를 통제한다.
```

예를 들어 코딩 에이전트가 아무리 우수해도 다음 조건이면 실패한다.

```
- 저장소 구조를 알 수 없다.
- 실행 명령이 문서화되어 있지 않다.
- 테스트가 없다.
- 로그를 읽을 수 없다.
- 운영 DB 와 개발 DB 가 분리되지 않았다.
- 완료 기준이 없다.
```

```
- 이전 실행의 진행상태가 남지 않는다.
- 에이전트가 모든 파일과 비밀정보에 접근한다.
```

이 경우 문제는 모델의 지능만이 아니라 **작업 환경이 에이전트에게 해석 가능하지 않다**는 데 있다. OpenAI 의 하네스 엔지니어링 사례 역시 사람이 코드를 직접 작성하는 역할에서 환경, 의도, 피드백 루프, 검증 장치를 설계하는 역할로 이동한다고 설명한다. (OpenAI, Harness engineering)

3.3 하네스의 9 개 계층

계층	역할	대표 산출물
1. Instruction	행동 규칙과 목표	AGENTS.md, Rules
2. Context	필요한 정보 제공	Architecture, Specs
3. Tools	실행 기능	Shell, Browser, MCP
4. Runtime	격리된 실행 환경	Sandbox, Container, Worktree
5. State	진행상태 보존	progress.md, Git, Task DB
6. Feedback	결과 검증	Test, Lint, Build, Logs
7. Orchestration	에이전트 역할 분담	Planner, Worker, Reviewer
8. Governance	권한과 위험 통제	Approval, Allowlist, Budget
9. Evaluation	시스템 성능 측정	Task suite, Trace, Grader

3.4 에이전트 친화적 저장소 구조

```
project/
├─ AGENTS.md
├─ README.md
├─ ARCHITECTURE.md
├─ SECURITY.md
│
├─ docs/
│  ├─ product-spec.md
│  ├─ repository-map.md
│  ├─ progress.md
│  ├─ decisions/
│  │  ├─ ADR-001-database.md
│  │  └─ ADR-002-authentication.md
│  └─ plans/
│     ├─ active/
│     └─ completed/
│
├─ src/
├─ tests/
│
├─ scripts/
│  ├─ setup.ps1
│  ├─ verify.ps1
│  ├─ smoke-test.ps1
│  └─ reset-test-data.ps1
```

```
│
├─ .github/
│  ├─ copilot-instructions.md
│  └─ instructions/
│
├─ .vscode/
│  └─ mcp.json
│
└─ artifacts/
   ├─ screenshots/
   ├─ logs/
   └─ test-results/
```

장기 작업에서는 대화창 자체를 기억장치로 사용하지 않는 것이 중요하다. Anthropic 의 장기 실행 에이전트 사례는 초기화 에이전트와 구현 에이전트를 구분하고, 진행상태 파일, Git 기록, 테스트 결과, 실행 스크립트 같은 외부 산출물을 다음 세션의 기억으로 사용한다. (Anthropic, Effective harnesses for long-running agents)

3.5 AGENTS.md 설계

AGENTS.md 는 모든 내용을 담는 거대한 지침서가 아니라, **저장소를 탐색하기 위한 짧은 지도**여야 한다. 세부 내용은 별도의 명세와 아키텍처 문서로 연결한다. 지나치게 큰 지침 파일은 컨텍스트를 소모하고 오래된 규칙을 계속 전달할 위험이 있다. (OpenAI, Harness engineering)

복사용 AGENTS.md

```
# AGENTS.md

## 1. Project Purpose
이 프로젝트는 대학원 수업의 학생, 과제, AI 활용 과정 및 평가를 관리하는
웹 애플리케이션이다.

## 2. Start Here
작업 전 다음 문서를 순서대로 읽는다.

1. `docs/product-spec.md`
2. `ARCHITECTURE.md`
3. `docs/repository-map.md`
4. `docs/progress.md`
5. 현재 작업과 관련된 `docs/decisions/` 문서

## 3. Repository Map
- `frontend/`: React + TypeScript
- `backend/`: FastAPI
- `backend/models/`: SQLAlchemy 모델
- `tests/`: 자동화 테스트
- `scripts/`: 설치·검증·테스트 스크립트

## 4. Invariants
다음 규칙은 임의로 변경하지 않는다.

- API 오류 형식은 `code`, `message`, `details`를 포함한다.
- DB 접근은 service layer 를 통한다.
- 비밀키를 코드나 문서에 기록하지 않는다.
- 기존 공개 API 를 변경하려면 별도 승인이 필요하다.
- 모든 사용자 입력은 서버에서 다시 검증한다.

## 5. Work Method
1. 수정 전에 관련 코드를 탐색한다.
2. 기존 패턴을 우선 재사용한다.
3. 한 번에 하나의 작은 작업만 구현한다.
4. 테스트를 먼저 추가하거나 실패를 재현한다.
```

````
5. 구현 후 대상 테스트와 전체 검증을 실행한다.
6. 작업 상태를 `docs/progress.md`에 기록한다.
7. 논리적으로 독립된 변경마다 Git 커밋을 만든다.

## 6. Verification
작업 완료 전 다음 명령을 실행한다.

```powershell
./scripts/verify.ps1
```

## 7. Permissions
사전 승인 없이 하지 않는 작업:

- 운영환경 배포
- 운영 DB 변경
- 데이터 삭제
- 인증·권한 정책 변경
- 외부 유료 API 호출
- 새 프레임워크 도입
- 주요 버전 의존성 업그레이드

## 8. Definition of Done
다음 조건을 모두 만족해야 완료이다.

- 요구사항과 완료 기준을 충족한다.
- 관련 테스트가 추가되었다.
- 전체 테스트, lint, type check, build 가 통과한다.
- 문서와 진행상태가 갱신되었다.
- 변경 파일과 검증 결과를 보고했다.
- 알려진 위험과 미완료 항목을 보고했다.

## 9. Escalation
다음 상황에서는 추측하여 진행하지 말고 중단한다.

- 요구사항이 서로 충돌한다.
- 데이터 손실 가능성이 있다.
- 보안 경계 변경이 필요하다.
- 두 번 이상 같은 실패가 반복된다.
- 성공 여부를 자동으로 검증할 방법이 없다.
````

3.6 완료 기준을 기계가 읽을 수 있게 만들기

```
{
  "task_id": "USER-014",
  "goal": "중복 이메일 사용자 등록을 409 오류로 처리한다.",
  "scope": [
    "backend 사용자 생성 서비스",
    "API 오류 매핑",
    "관련 테스트"
  ],
  "non_goals": [
    "로그인 기능 변경",
    "DB 스키마 변경",
    "프론트엔드 리디자인"
  ],
  "acceptance_criteria": [
    {
      "id": "AC-1",
      "condition": "새 이메일로 사용자를 생성하면",
      "expected": "201 과 사용자 정보를 반환한다."
    },
    {
      "id": "AC-2",
      "condition": "이미 존재하는 이메일로 생성하면",
      "expected": "409 와 DUPLICATE_EMAIL 코드를 반환한다."
    },
    {
      "id": "AC-3",
```

```
      "condition": "전체 테스트를 실행하면",
      "expected": "기존 테스트를 포함해 모두 통과한다."
    }
  ],
  "verification": [
    "pytest tests/test_users.py",
    "pytest",
    "ruff check .",
    "mypy backend"
  ],
  "requires_approval": [
    "DB schema change",
    "public API change",
    "new dependency"
  ]
}
```

3.7 검증 스크립트 예시

```
# scripts/verify.ps1

$ErrorActionPreference = "Stop"

Write-Host "1. Frontend lint"
npm --prefix frontend run lint

Write-Host "2. Frontend type check"
npm --prefix frontend run typecheck

Write-Host "3. Frontend tests"
npm --prefix frontend run test -- --run

Write-Host "4. Frontend build"
npm --prefix frontend run build

Write-Host "5. Backend lint"
python -m ruff check backend tests

Write-Host "6. Backend type check"
python -m mypy backend

Write-Host "7. Backend tests"
python -m pytest tests

Write-Host "All verification steps passed."
```

이 스크립트의 목적은 에이전트가 "잘 된 것 같다"고 판단하는 대신, 동일한 명령으로 완료 여부를 확인하게 하는 것이다.

3.8 하네스 실습

실습 주제

```
기존 To-do 애플리케이션에
"완료된 항목을 날짜별로 필터링하는 기능"을 추가한다.
```

1 단계: 기준선 측정

하네스 없이 한 번에 요청한다.

```
완료된 할 일을 날짜별로 필터링하는 기능을 추가해줘.
```

다음 항목을 기록한다.

```
- 첫 실행 성공 여부
- 수정 파일 수
- 테스트 통과 여부
- 기존 기능 회귀 여부
- 사람이 다시 수정한 횟수
- 모델이 잘못 가정한 내용
```

2 단계: 하네스 추가

```
- product-spec.md 작성
- AGENTS.md 작성
- 실패 테스트 작성
- verify.ps1 작성
- progress.md 작성
- 별도 Git 브랜치 또는 worktree 사용
- 브라우저 확인 절차 추가
```

3 단계: 동일 과제 재실행

두 실행 결과를 다음 기준으로 비교한다.

지표	기준선	하네스 적용
첫 실행 테스트 통과율		
회귀 오류 수		
사람 개입 횟수		
불필요한 변경 파일 수		
요구사항 누락 수		
완료 근거 제공 여부		

3.9 에이전트 하네스와 평가 하네스

두 개념을 구분해야 한다.

```
Agent Harness
= 에이전트가 일을 하게 하는 시스템

Evaluation Harness
= 에이전트가 일을 얼마나 잘했는지 측정하는 시스템
```

평가 하네스의 구성은 다음과 같다.

```
Task
실행할 문제

Trial
동일 문제를 반복한 개별 실행

Trace / Trajectory
모델 호출, 도구 호출, 관찰, 수정 과정

Outcome
최종 산출물과 환경 상태

Grader
```

정확성·안전성·완료 여부를 판정하는 평가자

Regression Suite
변경 후 기존 성능이 유지되는지 검사하는 문제 집합

---

4. 루프 엔지니어링

4.1 루프 엔지니어링의 개념

루프 엔지니어링은 아직 프롬프트 엔지니어링처럼 완전히 정착된 학술 용어는 아니다. 2026 년 들어 에이전트 작업을 반복적으로 운영하는 실무 개념으로 사용되기 시작했으며, 그 기반은 피드백 제어, ReAct, 평가자–개선자 패턴, CI/CD, 테스트 주도 개발, Human-in-the-loop 에 있다.

현재의 실무적 정의는 다음과 같다.

```
루프 엔지니어링이란
명확한 목표와 관찰 가능한 성공 기준을 설정하고,
에이전트가 행동한 결과를 평가하여,
다음 행동을 조정하고,
성공·실패·예산·위험 조건에 따라 종료하도록
반복 시스템을 설계하는 것이다.
```

일부 실무자들은 루프 엔지니어링을 하네스보다 한 단계 위에서, 일정·트리거·작업공간·하위 에이전트·영속 메모리를 결합하는 운영 설계로 설명한다. 이는 유용한 구분이지만 아직 유일한 표준 정의는 아니다. (Addy Osmani, Loop engineering)

4.2 단순 반복과 루프 엔지니어링의 차이

좋지 않은 반복:

```
될 때까지 계속 수정해.
```

이 경우 다음 문제가 생긴다.

```
- 무엇이 "됨"인지 불명확하다.
- 같은 해결책을 반복할 수 있다.
- 테스트에만 과적합할 수 있다.
- 변경 범위가 계속 커질 수 있다.
- 토큰과 시간이 무한히 사용될 수 있다.
- 데이터 삭제나 배포 같은 위험한 행동을 할 수 있다.
```

엔지니어링된 루프:

```
Goal
+ State
+ Action
+ Observation
+ Evaluator
+ Stop Condition
+ Budget
+ Approval Gate
+ Escalation
```

4.3 제어공학 관점의 루프

제어 시스템	AI 에이전트 시스템
목표값·Setpoint	완료 기준

제어 시스템	AI 에이전트 시스템
Controller	에이전트
Actuator	도구·MCP
Plant	코드베이스·브라우저·DB
Sensor	테스트·로그·메트릭
Error Signal	기대 결과와 실제 결과의 차이
Feedback	테스트 실패·리뷰 의견
Stability	반복이 수렴하는가
Stop Condition	오차가 허용범위 이내인가

따라서 루프 엔지니어링의 본질은 "많이 반복시키는 것"이 아니라 **오차가 줄어드는 폐쇄형 피드백 구조**를 만드는 것이다.

4.4 기본 루프 의사코드

```
state = load_persistent_state()
history = []
MAX_ITERATIONS = 5

for iteration in range(1, MAX_ITERATIONS + 1):
    goal = state.goal
    acceptance = state.acceptance_criteria

    observation = inspect_environment(state)

    next_action = agent_choose_smallest_action(
        goal=goal,
        state=state,
        observation=observation,
        previous_failures=history
    )

    if next_action.requires_approval:
        escalate_to_human(next_action)
        break

    result = execute_in_sandbox(next_action)

    evaluation = run_validators(
        result=result,
        acceptance_criteria=acceptance
    )

    record_trace(
        iteration=iteration,
        action=next_action,
        result=result,
        evaluation=evaluation
    )

    if evaluation.all_passed:
        save_state(status="completed")
        break

    failure_fingerprint = create_failure_fingerprint(evaluation)

    if repeated_without_new_evidence(
        failure_fingerprint,
        history,
```

```
        threshold=2
    ):
        save_state(status="blocked")
        escalate_to_human("동일 실패가 반복되었음")
        break

    state = update_state(
        state,
        observation,
        result,
        evaluation
    )

    history.append(failure_fingerprint)

else:
    save_state(status="budget_exceeded")
    escalate_to_human("최대 반복 횟수를 초과함")
```

4.5 루프 명세 예시

```
loop:
  id: duplicate-email-fix

  goal:
    description: "중복 이메일 등록 시 500 이 아닌 409 를 반환한다."

  state:
    file: "docs/progress.md"
    source_of_truth: "docs/product-spec.md"

  allowed_actions:
    - read_repository
    - search_code
    - edit_backend
    - edit_tests
    - run_targeted_tests
    - run_full_verification

  validators:
    targeted:
      - "pytest tests/test_users.py::test_duplicate_email"
    regression:
      - "pytest"
      - "ruff check ."
      - "mypy backend"

  iteration_policy:
    max_iterations: 5
    one_hypothesis_per_iteration: true
    maximum_changed_files_per_iteration: 4
    require_test_before_fix: true

  stop_when:
    - "targeted test passes"
    - "full regression passes"
    - "acceptance criteria all pass"

  stop_on:
    - "same failure fingerprint occurs twice"
    - "public API change is required"
    - "database migration is required"
    - "security boundary change is required"
    - "verification cannot be automated"

  approval_required:
    - dependency_install
    - file_delete
    - database_schema_change
    - network_write
    - deployment
```

```
output:
  - changed_files
  - root_cause
  - test_evidence
  - remaining_risks
```

4.6 대표적인 루프 유형

루프	관찰 자료	사용 사례
Test-driven loop	테스트 결과	버그 수정
Compiler loop	컴파일·타입 오류	코드 구현
Lint loop	정적 분석 결과	품질 개선
Browser loop	DOM·스크린샷·콘솔	UI 개발
Evaluator loop	평가자 점수	문서·콘텐츠 개선
Security loop	보안 스캔 결과	취약점 수정
Performance loop	프로파일·메트릭	성능 최적화
Review loop	PR 리뷰 의견	코드 리뷰 반영
Maintenance loop	이슈·로그·알림	정기 유지보수

4.7 버그 수정 루프용 프롬프트

```
# 역할
당신은 테스트 기반 디버깅 에이전트이다.

# 목표
실패를 재현하고 근본 원인을 최소 변경으로 수정한다.

# 작업 규칙
1. 먼저 실패 테스트를 직접 실행한다.
2. 실패를 재현하기 전에는 코드를 수정하지 않는다.
3. 오류 메시지, 스택 트레이스, 관련 코드 경로를 확인한다.
4. 한 반복에서 하나의 원인 가설만 검증한다.
5. 수정 후 가장 좁은 대상 테스트를 먼저 실행한다.
6. 대상 테스트가 통과하면 전체 검증을 실행한다.
7. 테스트를 삭제하거나 완화하여 통과시키지 않는다.
8. 공개 API 나 DB 스키마 변경이 필요하면 중단한다.
9. 같은 실패가 새로운 증거 없이 두 번 반복되면 중단한다.
10. 최대 반복 횟수는 5 회이다.

# 각 반복의 보고 형식
- 반복 번호:
- 관찰된 실패:
- 원인 가설:
- 검증 방법:
- 변경 파일:
- 테스트 결과:
- 다음 결정:

# 완료 보고
- 근본 원인
- 변경 내용
- 대상 테스트 결과
- 전체 테스트 결과
- 남아 있는 위험
```

4.8 루프 엔지니어링의 주요 실패

Thrashing

서로 반대되는 수정을 반복한다.

```
A 방식으로 수정
→ 테스트 1 통과, 테스트 2 실패
→ B 방식으로 되돌림
→ 테스트 2 통과, 테스트 1 실패
```

대책:

```
- 실패 지문 저장
- 이전 가설과 결과 기록
- 동일 가설 재시도 금지
- 구조적 원인 분석 단계 추가
```

테스트 과적합

특정 테스트만 통과하도록 하드코딩한다.

대책:

```
- 대상 테스트와 전체 회귀 테스트 분리
- 속성 기반 테스트
- 경계값 테스트
- 숨겨진 평가 사례
- 테스트 코드 변경 시 별도 승인
```

컨텍스트 드리프트

반복하면서 원래 목표가 흐려진다.

대책:

```
- 매 반복마다 목표와 비범위를 다시 읽는다.
- 현재 상태는 progress.md 에 기록한다.
- 완료 기준을 변경하려면 승인받는다.
```

위험한 자율성

에이전트가 배포·삭제·결제까지 자동으로 수행한다.

대책:

```
- 읽기 → 로컬 쓰기 → 브랜치 → PR → 배포의 권한 단계를 구분한다.
- 외부 부작용이 있는 작업은 사람 승인 단계로 둔다.
- 운영 자격증명을 에이전트 환경에 기본 제공하지 않는다.
```

5. 바이브 코딩에서의 엔지니어링 기법

5.1 바이브 코딩의 정확한 위치

바이브 코딩은 자연어를 주요 인터페이스로 사용해 코드를 생성·수정하는 개발 방식이다. 그러나 자연어로 요청한다고 자동으로 엔지니어링이 되는 것은 아니다.

```
바이브 코딩
= 개발 인터페이스

프롬프트·컨텍스트·에이전트·하네스·루프 엔지니어링
= 개발 품질을 만드는 운영 체계
```

바이브 코딩의 성숙도

```
Level 0. Chatty Coding
"앱을 만들어줘."

Level 1. Prototype Coding
기능 목록을 주고 빠르게 시제품을 만든다.

Level 2. Spec-driven Coding
명세와 비범위, 완료 기준을 먼저 작성한다.

Level 3. Plan-driven Coding
저장소 조사와 구현 계획을 승인한 뒤 개발한다.

Level 4. Test-driven Coding
실패 테스트와 검증을 중심으로 개발한다.

Level 5. Harnessed Coding
규칙, 문서, 도구, 상태, 권한, 테스트가 저장소에 구축된다.

Level 6. Loop-engineered Development
에이전트가 반복적으로 구현·검증하되,
명시적 중단·예산·승인 조건 안에서 움직인다.
```

실무 프로젝트는 최소한 Level 3~5 를 목표로 해야 한다.

5.2 개발을 시작하는 올바른 순서

```
아이디어
  ↓
문제와 사용자 정의
  ↓
기능 범위·비범위 정의
  ↓
완료 기준 작성
  ↓
기존 저장소 조사
  ↓
기술적 위험과 미확정 사항 분석
  ↓
아키텍처와 데이터 계약
  ↓
구현 계획
  ↓
가장 얇은 수직 기능 구현
  ↓
자동 테스트·브라우저 검증
  ↓
리뷰·문서화
```

```
  ↓
다음 기능
```

핵심은 처음부터 “전체 앱을 완성하라”고 요청하지 않는 것이다.

5.3 개발 시작용 마스터 프롬프트

```
# 역할
당신은 이 저장소의 구현 에이전트이다.
기존 아키텍처와 규칙을 존중하면서 검증 가능한 작은 변경을 수행한다.

# 제품 목표
[제품이 해결할 문제와 사용자]

# 이번 목표
[이번 세션에서 완성할 하나의 기능]

# 현재 상태
- 저장소 위치:
- 실행 방법:
- 이미 구현된 기능:
- 알려진 문제:
- 관련 문서:

# 범위
이번 작업에 포함:
1.
2.
3.

이번 작업에 포함하지 않음:
1.
2.
3.

# 필수 제약
- 기존 공개 API 를 임의로 변경하지 않는다.
- 기존 코드 패턴을 우선 사용한다.
- 새 의존성을 추가하기 전에 필요성을 보고한다.
- 비밀정보를 읽거나 출력하지 않는다.
- 운영환경에 접근하지 않는다.
- 한 번에 하나의 작은 수직 기능만 구현한다.

# Phase 0: 저장소 조사
아직 코드를 수정하지 않는다.

1. AGENTS.md 와 프로젝트 문서를 읽는다.
2. 저장소 구조를 파악한다.
3. 관련 코드와 테스트를 찾는다.
4. 기존 구현 패턴을 설명한다.
5. 요구사항의 모호함과 위험을 정리한다.
6. 수정이 필요한 파일 후보를 제시한다.

다음 형식으로만 보고한다.
- 현재 구조
- 관련 파일
- 기존 패턴
- 미확정 사항
- 위험
- 구현 후보

# Phase 1: 계획
코드를 수정하지 않고 다음 내용을 작성한다.

1. 기능 동작
2. 데이터 흐름
3. API 또는 컴포넌트 계약
4. 단계별 구현 계획
```

```
5. 각 단계의 검증 명령
6. 롤백 방법

# Phase 2: 구현
계획에 따라 가장 작은 단계 하나만 구현한다.

각 단계마다:
1. 변경 전 실패를 재현한다.
2. 최소 변경을 수행한다.
3. 대상 테스트를 실행한다.
4. 전체 회귀 검증을 실행한다.
5. progress.md 를 갱신한다.

# 완료 기준
- [구체적 사용자 행동]
- [API 또는 데이터 기준]
- [테스트 기준]
- [접근성·성능·보안 기준]
- [문서화 기준]

# 중단 조건
- 요구사항 충돌
- DB 마이그레이션 필요
- 공개 API 파괴 필요
- 인증·권한 변경 필요
- 같은 실패 2 회 반복
- 검증 수단 없음

# 최종 보고
1. 구현 요약
2. 변경 파일과 이유
3. 테스트 명령과 결과
4. 스크린샷 또는 로그 증거
5. 알려진 제한
6. 다음 작업
```

5.4 기능 개발 요청

```
다음 기능을 하나의 수직 기능 단위로 구현하라.

# 사용자 스토리
[사용자]는 [목적]을 위해 [행동]할 수 있어야 한다.

# 기능
학생이 과제를 제출하면 교수자가 제출 상태와 제출 시간을 확인할 수 있어야 한다.

# 완료 기준
1. 학생은 PDF 파일 하나를 업로드할 수 있다.
2. 허용 확장자가 아니면 저장하지 않는다.
3. 20MB 를 초과하면 명확한 오류를 표시한다.
4. 제출 시간은 서버 시간 기준으로 저장한다.
5. 교수자 화면에서 제출 여부와 시간이 표시된다.
6. 권한 없는 사용자는 다른 학생의 파일을 볼 수 없다.
7. 성공·실패·권한·경계값 테스트가 통과한다.

# 구현 원칙
- 먼저 현재 업로드 패턴을 조사한다.
- DB 부터 UI 까지 연결되는 얇은 기능을 구현한다.
- 전체 화면을 재설계하지 않는다.
- 기존 API 형식을 유지한다.
- 파일 저장 경로를 사용자 입력으로 직접 구성하지 않는다.

구현 전에 관련 파일, 데이터 흐름, 위험, 테스트 계획을 먼저 보고하라.
```

5.5 버그 수정 요청

```
다음 오류를 수정하라.

# 기대 동작
중복 이메일로 가입하면 409 DUPLICATE_EMAIL 을 반환해야 한다.

# 실제 동작
500 Internal Server Error 가 발생한다.

# 재현 절차
1. POST /users 로 user@example.com 을 등록한다.
2. 같은 이메일로 다시 요청한다.
3. 500 응답이 발생한다.

# 규칙
- 먼저 오류를 재현한다.
- 스택 트레이스와 DB 오류를 확인한다.
- 실패 테스트를 추가한다.
- 근본 원인을 설명한 뒤 최소 변경한다.
- 테스트를 약화하거나 예외를 무시하지 않는다.
- 전체 테스트를 실행한다.

# 보고
1. 재현 결과
2. 근본 원인
3. 수정 파일
4. 대상 테스트
5. 전체 회귀 결과
6. 남은 위험
```

5.6 리팩터링 요청

```
다음 코드를 외부 동작을 변경하지 않고 리팩터링하라.

# 목적
사용자 검증 로직이 route, service, model 에 중복되어 있어 유지보수가 어렵다.

# 반드시 유지할 불변조건
- API 경로와 응답 스키마
- 오류 코드
- DB 스키마
- 기존 테스트의 기대값
- 권한 정책

# 작업 절차
1. 현재 동작을 테스트로 고정한다.
2. 중복 코드를 위치별로 정리한다.
3. 목표 구조를 제안한다.
4. 한 단계씩 이동한다.
5. 매 단계마다 테스트를 실행한다.
6. 성능이나 의존성 변화가 있으면 보고한다.

# 완료 기준
- 중복 검증 로직이 하나의 계층에 집중된다.
- 외부 동작이 변경되지 않는다.
- 전체 테스트가 통과한다.
- 새 구조와 책임이 문서화된다.
```

5.7 UI·UX 개발 요청

```
# 목표
학생 평가 화면의 정보 밀도를 줄이고,
```

```
교수자가 미평가 과제를 빠르게 찾도록 개선한다.

# 사용자 주요 과업
1. 미제출 학생 확인
2. 제출했지만 미평가인 학생 확인
3. 평가 완료 학생 확인
4. 학생별 제출물 열기
5. 루브릭 점수 입력

# UX 원칙
- 상태는 색상만으로 구분하지 않는다.
- 미평가 항목을 시각적으로 우선시한다.
- 주요 동작은 한 화면에서 발견 가능해야 한다.
- 파괴적 동작은 보조 버튼으로 배치한다.
- 기존 디자인 시스템 컴포넌트를 재사용한다.
- 키보드 탐색과 스크린리더 레이블을 제공한다.

# 구현 절차
1. 기존 화면과 사용자 흐름을 조사한다.
2. 현재 문제를 사용자 과업 기준으로 분류한다.
3. 구현 전에 와이어 구조를 텍스트로 제안한다.
4. 승인된 구조로 UI 를 구현한다.
5. 브라우저에서 데스크톱과 모바일을 확인한다.
6. 콘솔 오류, 접근성, 빈 상태, 오류 상태를 검증한다.

# 완료 증거
- 변경 전후 화면
- 주요 사용자 흐름
- 접근성 검사 결과
- 브라우저 콘솔 결과
- 관련 컴포넌트 테스트
```

---

6. Cursor, VS Code, Antigravity 의 활용 방법

여기서 Antigravity 는 **Google Antigravity** 를 의미하는 것으로 정리한다.

6.1 비교

도구	중심 성격	적합한 상황	핵심 설정
Cursor	코드 편집 중심 AI IDE	저장소 안에서 빠른 계획–수정–검증	Rules, AGENTS.md, Plan Mode, Skills, MCP
VS Code + Copilot	기존 IDE 생태계에 에이전트 기능 통합	기존 확장·디버깅·조직 표준 유지	Instructions, Agents, Prompt files, Skills, MCP
Google Antigravity	에이전트 우선 다중 작업·다중 프로젝트 환경	여러 에이전트·브라우저·스케줄 작업 통합	Projects, Artifacts, Permissions, MCP, Subagents

Cursor 공식 문서는 Agent, Plan Mode, 프로젝트 규칙과 AGENTS.md, Skills, MCP, 하위 에이전트, 격리된 worktree 와 hooks 를 제공하는 흐름을 설명한다. (Cursor documentation)

VS Code 의 현재 AI 개발 환경은 코드 중심 Chat View 와 여러 에이전트 세션을 관리하는 Agents Window 를 구분하고, 프로젝트 지침, AGENTS.md, 지시 파일, Skills, MCP, 체크포인트, Chat Debug 를 지원한다. (Visual Studio Code, Chat agent mode)

Google Antigravity 2.0 은 여러 로컬 에이전트와 프로젝트를 관리하는 에이전트 중심 개발 플랫폼으로 소개되고 있으며, IDE 뿐 아니라 CLI 와 SDK, Artifacts, 브라우저·터미널 작업, 동적 하위 에이전트, 예약 작업, 프로젝트 범위 권한과 MCP 설정을 제공한다. (Google Antigravity)

6.2 Cursor 권장 흐름

1. 저장소를 연다.
2. AGENTS.md 또는 Project Rules 를 작성한다.
3. Plan Mode 에서 저장소를 조사하게 한다.
4. 곧바로 코딩하지 않고 구현 계획을 검토한다.
5. 하나의 수직 기능만 선택한다.
6. 별도 브랜치 또는 worktree 에서 작업한다.
7. 필요한 MCP 만 프로젝트 단위로 연결한다.
8. 테스트, lint, type check, build 명령을 규칙에 기록한다.
9. 복잡한 작업만 하위 에이전트로 분리한다.
10. diff 와 테스트 증거를 확인한 후 병합한다.

Cursor 시작 프롬프트

```
Plan Mode 로 작업하라.

아직 코드를 수정하지 말고 다음을 수행한다.

1. AGENTS.md 와 관련 문서를 읽는다.
2. 현재 저장소 구조를 설명한다.
3. 요청한 기능과 관련된 파일을 찾는다.
4. 기존 구현 패턴과 테스트 패턴을 조사한다.
5. 수정이 필요한 파일 후보를 제시한다.
6. 가장 작은 수직 구현 계획을 작성한다.
7. 각 단계의 검증 명령을 포함한다.
8. 위험한 변경과 승인이 필요한 사항을 표시한다.

계획 승인 전에는 파일을 수정하지 않는다.
```

6.3 VS Code + Copilot 권장 흐름

1. 프로젝트 지침을 초기화한다.
2. `.github/copilot-instructions.md` 또는 AGENTS.md 를 작성한다.
3. 특정 파일 패턴에는 `.instructions.md`를 사용한다.
4. 반복 작업은 prompt file 이나 Skill 로 분리한다.
5. 코드 중심 작업은 Chat View 에서 수행한다.
6. 여러 프로젝트·세션 작업은 Agents Window 에서 관리한다.
7. `.vscode/mcp.json`에 최소한의 MCP 서버만 설정한다.
8. 체크포인트와 Git diff 로 변경을 검토한다.
9. Chat Debug 에서 실제 컨텍스트와 도구 호출을 확인한다.
10. 테스트가 완료된 변경만 PR 로 이동한다.

VS Code 지침 파일 예시

```
---
applyTo: "backend/**/*.py"
---

# Backend Python Instructions

- FastAPI route 는 입력 검증과 응답 매핑만 담당한다.
- 비즈니스 로직은 service layer 에 둔다.
- DB 접근은 repository layer 를 통한다.
- 모든 공개 함수에는 타입 힌트를 작성한다.
- 새 API 에는 성공, 입력 오류, 권한, 경계값 테스트를 추가한다.
```

```
- 예외를 일반 Exception 으로 삼키지 않는다.
- API 오류는 프로젝트 표준 형식을 사용한다.
```

6.4 Google Antigravity 권장 흐름

Antigravity 는 편집기 한 화면에서 한 에이전트에게 요청하는 방식보다, 여러 에이전트와 프로젝트를 운영하는 방식에 더 적합하다.

```
1. 하나의 제품 단위를 Project 로 정의한다.
2. 프로젝트가 참조할 폴더와 저장소를 명시한다.
3. 프로젝트 범위 권한과 MCP allowlist 를 설정한다.
4. 먼저 Artifact 로 요구사항·계획·검증 기준을 생성한다.
5. Planner, Implementer, Reviewer 역할을 구분한다.
6. 코드·터미널·브라우저 작업을 서로 다른 하위 에이전트에 배치한다.
7. 결과는 대화가 아니라 Artifact 와 Git 변경으로 공유한다.
8. 브라우저 에이전트로 실제 사용자 흐름을 검증한다.
9. 안정된 작업만 예약 실행으로 전환한다.
10. 배포·데이터 변경은 사람 승인 단계로 유지한다.
```

Antigravity 다중 에이전트 요청 예시

```
이 작업을 세 역할로 분리한다.

Planner Agent
- 저장소와 요구사항 분석
- 관련 파일과 위험 식별
- 완료 기준과 단계별 계획 작성
- 코드는 수정하지 않음

Implementation Agent
- 승인된 계획의 첫 번째 수직 기능만 구현
- 관련 테스트 추가
- 변경 범위를 제한
- 진행상태 Artifact 갱신

Reviewer Agent
- 요구사항 누락 검토
- 보안·회귀·접근성 검토
- 테스트 결과와 브라우저 결과 확인
- 직접 수정하지 않고 우선 리뷰 보고서 작성

공유 상태는 대화 기록이 아니라 다음 산출물을 사용한다.
- Product Spec Artifact
- Active Plan Artifact
- Progress Artifact
- Git diff
- Test Result Artifact
```

---

7. 바이브 코딩을 실제 엔지니어링으로 만드는 운영 규칙

7.1 반드시 분리해야 하는 단계

```
탐색과 수정은 분리한다.
계획과 구현은 분리한다.
구현과 검증은 분리한다.
작성자와 평가자는 가능하면 분리한다.
로컬 변경과 외부 부작용은 분리한다.
```

7.2 한 번에 전체 앱을 요청하지 않는다

좋지 않은 방식:

```
쇼핑몰 전체를 만들어줘.
로그인, 결제, 관리자, 추천, 배포까지 다 해줘.
```

개선된 방식:

```
1 단계: 상품 목록을 읽기 전용으로 표시
2 단계: 상품 상세
3 단계: 로컬 장바구니
4 단계: 사용자 인증
5 단계: 서버 장바구니
6 단계: 주문 생성
7 단계: 결제 샌드박스
8 단계: 관리자 주문 조회
```

각 단계는 DB–API–UI–테스트가 연결되는 **얇은 수직 기능**이어야 한다.

7.3 AI 에게 맡겨도 되는 것과 승인해야 하는 것

자율 실행 가능	사람 승인 필요
코드 검색	운영 배포
파일 읽기	운영 DB 변경
로컬 테스트	데이터 삭제
테스트 코드 추가	인증 정책 변경
별도 브랜치 수정	외부 결제
문서 초안	사용자에게 이메일 발송
lint·type check	비밀키·권한 변경
로컬 브라우저 검증	주요 의존성 업그레이드

7.4 바이브 코딩 완료 보고서

```
# 작업 결과

## 1. 목표
이번 작업에서 해결한 문제:

## 2. 구현 내용
- 기능:
- 변경된 사용자 흐름:
- 변경된 API:
- 변경된 데이터 구조:

## 3. 변경 파일
| 파일 | 변경 이유 |
|---|---|

## 4. 검증
- 대상 테스트:
- 전체 테스트:
- lint:
```

```
- type check:
- build:
- 브라우저:
- 접근성:

## 5. 완료 기준
| 기준 | 결과 | 근거 |
|---|---|---|

## 6. 미완료 및 위험
- 알려진 제한:
- 추후 작업:
- 보안 검토 필요사항:
- 성능 검토 필요사항:

## 7. 재현 방법
1.
2.
3.
```

---

8. 수업용 4 단계 실습 과정

1 단계: 프롬프트와 평가

목표

같은 과제를 단순 프롬프트와 구조화 프롬프트로 실행하고
결과의 정확성과 일관성을 비교한다.

산출물

- 기준 프롬프트
- 개선 프롬프트
- 평가 사례 10 개
- 평가표
- 실패 유형 분석

---

2 단계: 에이전트와 MCP

목표

읽기 전용 MCP 도구를 통해
에이전트가 로컬 문서 또는 저장소를 검색하고 분석하게 한다.

산출물

- 에이전트 명세
- 도구 목록과 스키마
- MCP 설정
- 권한표
- 도구 호출 로그
- 프롬프트 인젝션 대응 규칙

---

3 단계: 하네스 구축

목표

AGENTS.md, 명세, 진행상태, 테스트, 검증 스크립트,
별도 브랜치를 갖춘 에이전트 친화적 저장소를 만든다.

산출물

- AGENTS.md
- product-spec.md
- repository-map.md
- progress.md
- verify.ps1
- 테스트
- 완료 기준 JSON

---

4 단계: 루프 기반 바이브 코딩

목표

```
실패 테스트를 기준으로 최대 5 회의 수정 루프를 실행하고,
성공·중단·승인 조건에 따라 에이전트를 통제한다.
```

산출물

```
- loop.yaml
- 반복별 Trace
- 실패 지문
- 최종 Git diff
- 테스트 증거
- 사람 개입 기록
- 사후 분석
```

---

9. 학생 AI 활용 평가 기준

학생의 AI 활용 능력은 "결과물이 그럴듯한가"만으로 평가하면 안 된다. **AI 에게 어떤 일을 맡겼는지, 어떤 정보를 제공했는지, 어떻게 검증했는지, 실패를 어떻게 통제했는지**를 함께 평가해야 한다.

평가영역	평가 내용	배점
문제 정의	사용자·문제·목표가 명확한가	10
프롬프트 설계	역할·범위·제약·출력 형식이 명확한가	10
컨텍스트 설계	필요한 자료를 선별·구조화했는가	10
에이전트 설계	상태·행동·도구·중단 조건이 있는가	10
MCP·도구 설계	최소 권한과 명확한 스키마를 적용했는가	10
하네스 구축	규칙·상태·검증·복구 체계가 있는가	15
루프 설계	평가·재시도·예산·중단 조건이 적절한가	15
결과 검증	테스트·로그·스크린샷 등 근거가 있는가	10
이해와 설명	생성된 코드와 구조를 학생이 설명할 수 있는가	5
윤리·보안	개인정보·저작권·권한을 고려했는가	5
합계		100

성적 판정

```
A+  95~100
- 요구사항부터 검증까지 전 과정이 재현 가능하다.
- 하네스와 루프가 실제 오류를 탐지하고 중단한다.
- 학생이 모든 주요 설계 결정을 설명할 수 있다.

A   90~94
- 프롬프트, 컨텍스트, 테스트, 권한 설계가 충실하다.
- 일부 수작업이 있으나 전체 과정이 안정적이다.

B+  85~89
```

```
- 기능은 완성되었고 기본 검증이 있다.
- 상태관리, 중단조건 또는 보안 설계가 다소 부족하다.

B  80~84
- AI 를 활용해 결과는 만들었으나
  프롬프트와 검증 절차가 부분적으로만 기록되어 있다.

C+  75~79
- 결과는 동작하지만 테스트와 완료 기준이 부족하다.
- 학생이 생성된 코드 일부를 충분히 설명하지 못한다.

C  70~74
- 단순 대화형 바이브 코딩 수준이다.
- 결과를 검증하거나 반복을 통제하는 체계가 거의 없다.

D  60~69
- AI 산출물을 그대로 제출했으며
  오류·보안·저작권·의존성에 대한 검토가 부족하다.

F  0~59
- 결과가 재현되지 않는다.
- 학생이 코드와 설계를 설명하지 못한다.
- 허위 검증 결과, 출처 미표기, 심각한 보안 위반이 있다.
```

---

10. 최종 정리

```
프롬프트 엔지니어링
= 좋은 지시를 설계하는 기술

컨텍스트 엔지니어링
= 모델이 현재 알아야 할 정보를 설계하는 기술

워크플로 엔지니어링
= 정해진 처리 단계를 설계하는 기술

에이전트 엔지니어링
= 모델이 다음 행동과 도구를 선택하게 만드는 기술

MCP
= 에이전트와 외부 시스템을 연결하는 표준 규격

하네스 엔지니어링
= 에이전트의 지침, 환경, 권한, 상태, 검증, 복구를 설계하는 기술

루프 엔지니어링
= 목표 달성까지 실행–관찰–평가–수정을 반복하고
  성공·실패·예산·위험 조건에 따라 종료시키는 기술

바이브 코딩
= 자연어를 개발 인터페이스로 사용하는 방식

엔지니어링된 바이브 코딩
= 명세 + 계획 + 컨텍스트 + 도구 + 테스트
  + 하네스 + 피드백 루프 + 사람의 최종 책임
```

가장 중요한 변화는 다음과 같다.

```
과거:
사람이 코드를 작성하고 AI 가 일부를 보조한다.

현재:
사람이 목표, 경계, 환경, 검증 기준을 설계하고
AI 에이전트가 구현과 반복을 수행한다.

따라서 앞으로 중요한 능력은
```

"프롬프트를 잘 쓰는 능력" 하나가 아니라,

1. 문제를 정의하는 능력
2. 필요한 컨텍스트를 선별하는 능력
3. 도구와 권한을 설계하는 능력
4. 검증 가능한 환경을 만드는 능력
5. 실패가 수렴하도록 루프를 설계하는 능력
6. 최종 결과를 이해하고 책임지는 능력

이다.

온라인 출처 목록

1. OpenAI. *Prompt engineering*. https://developers.openai.com/api/docs/guides/prompt-engineering
2. Wei, J., et al. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. https://arxiv.org/abs/2201.11903
3. Anthropic. *Building effective agents*. https://www.anthropic.com/engineering/building-effective-agents
4. Anthropic. *Effective context engineering for AI agents*. https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents
5. OpenAI. *OpenAI Agents SDK: Running agents*. https://openai.github.io/openai-agents-python/running_agents/
6. Model Context Protocol. *Introduction*. https://modelcontextprotocol.io/docs/getting-started/intro
7. Visual Studio Code. *MCP servers*. https://code.visualstudio.com/docs/agent-customization/mcp-servers
8. Model Context Protocol. *Security best practices*. https://modelcontextprotocol.io/docs/tutorials/security/security_best_practices
9. Anthropic. *Demystifying evals for AI agents*. https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents
10. OpenAI. *Harness engineering*. https://openai.com/index/harness-engineering/
11. Anthropic. *Effective harnesses for long-running agents*. https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents
12. Osmani, A. *Loop engineering*. https://addyosmani.com/blog/loop-engineering/
13. Cursor. *Documentation*. https://cursor.com/docs
14. Visual Studio Code. *Chat agent mode*. https://code.visualstudio.com/docs/copilot/chat/chat-agent-mode
15. Google. *Antigravity*. https://antigravity.google/